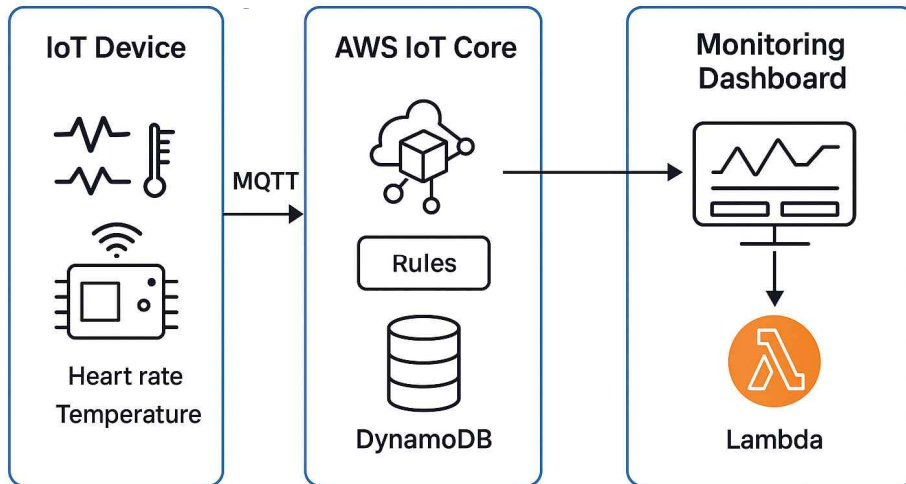


디지털 바이오 구현 : 연결 환자의 생체 데이터 모니터링 구현

아래는 **AWS IoT 기반 디지털 바이오 시스템 구현 예제**로, 연결된 환자의 생체 데이터를 모니터링하는 간단한 구조를 구현하는 내용이다.



구현 목표

- AWS IoT Core에 생체 센서 데이터를 MQTT로 전송
- AWS IoT Rules를 통해 데이터를 Amazon DynamoDB 또는 AWS Lambda로 전송
- 환자의 실시간 생체 정보 모니터링 대시보드 구현 (예: Amazon CloudWatch, 또는 웹앱)

구현 시나리오

ESP32 디바이스가 환자의 **심박수**와 **체온** 데이터를 5초마다 MQTT로 전송한다.

AWS IoT Core가 메시지를 받아 **DynamoDB**에 저장한다.

간단한 **웹 대시보드** 또는 **CloudWatch Dashboard**를 통해 실시간으로 모니터링한다.

1. 준비 사항

하드웨어 (모의 또는 실제)

- ESP32 또는 Raspberry Pi (MQTT client 역할)
- 또는 Python 기반 시뮬레이터로 대체
-

AWS 서비스

- AWS IoT Core
- AWS IAM (권한 설정)
- AWS DynamoDB
- AWS Lambda (옵션)
- Amazon CloudWatch (옵션)
- Amazon API Gateway + Lambda + HTML (웹 대시보드 용도)

2. IoT 디바이스 → MQTT 데이터 전송 예제 (Python 기반 시뮬레이터)

<코드 샘플>

```
import time
import random
import json
import paho.mqtt.client as mqtt

# AWS IoT Core endpoint
endpoint = "your-iot-endpoint.amazonaws.com"
port = 8883
topic = "bio/patient1"

# Load certificates (for TLS)
ca_path = "./AmazonRootCA1.pem"
cert_path = "./device-certificate.pem.crt"
key_path = "./private.pem.key"
```

```

client = mqtt.Client()
client.tls_set(ca_certs=ca_path, certfile=cert_path, keyfile=key_path)
client.connect(endpoint, port)

while True:
    payload = {
        "patient_id": "patient1",
        "timestamp": int(time.time()),
        "heart_rate": random.randint(60, 100),
        "temperature": round(random.uniform(36.5, 38.0), 1)
    }
    client.publish(topic, json.dumps(payload), qos=1)
    print(f"Published: {payload}")
    time.sleep(5)

```

3. AWS IoT Core 설정

IoT Thing & Certificate 등록

1. AWS IoT 콘솔 > **Thing** 생성
2. 인증서 생성 및 위의 Python 코드에 사용

IoT Core Rule 생성

- **SQL:**

```
SELECT * FROM 'bio/patient1'
```

- **동작:**
 - **DynamoDB** 테이블에 쓰기
 - 또는 **Lambda**로 전달

4. DynamoDB 테이블 생성

- 테이블 이름: PatientVitals

- 파티션 키: patient_id (String)
- 정렬 키: timestamp (Number)

5. Lambda (옵션 - 데이터 처리)

<코드 샘플>

```
import boto3
import json
```

```
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('PatientVitals')
```

```
def lambda_handler(event, context):
    for record in event['records']:
        payload = json.loads(record['value'])
        table.put_item(Item=payload)
    return {'statusCode': 200}
```

Lambda를 사용하지 않아도, IoT Rule만으로 DynamoDB에 직접 저장 가능

6. 데이터 시각화

📌 Option 1: CloudWatch Dashboard

- Lambda를 통해 Metric을 CloudWatch로 전송
- CloudWatch에서 환자별 그래프 구성

📌 Option 2: 간단한 웹 대시보드

- API Gateway + Lambda → DynamoDB 쿼리
- HTML/JavaScript로 실시간 갱신 UI 구성

7. 요약

구성 요소	역 할
ESP32 / 시뮬레이터	생체 정보 MQTT 전송
AWS IoT Core	메시지 수신 및 라우팅
DynamoDB	데이터 저장
Lambda (옵션)	데이터 가공 및 알람 트리거 등 처리
CloudWatch/Web	실시간 모니터링

이 실습은 교육용 또는 PoC(Proof of Concept, 소규모로 검증해보는 단계) 환경에 적합하다. 실제 환자 데이터를 사용하는 경우 **의료기기 규정, 보안, 개인정보보호법** 등을 철저히 준수해야 한다.